SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code:	CSA10 / CSA1005	**Course Title:**	Software Engineering
CO Assessed:	CO1	**Assessment:**	Assessment Tool 1 – Case Study Analysis
Weightage:	20%	**Total Marks:**	25
CO1 Statement:	*Apply advanced methodologies and agile project management techniques to manage software projects effectively*		
Student Name:	M. Rudresh	**Reg. No.:**	192372339
Date:	10-10-2026	**Duration:**	60 minutes

Code of Conduct

I M. Rudresh / 192372339 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Case Study Analysis

Instructions to Students:

Each student will be allotted **one problem statement**. Analyze the assigned problem systematically by following the steps given below. Your report should demonstrate your understanding of software engineering concepts and your ability to design an appropriate software solution.

Based on your allotted problem statement, prepare a report by answering the following questions:

1. **Understand the Problem Statement**
   - Explain the problem in your own words.
   - Identify the objectives of the proposed system.
   - State the scope of the problem.
2. **Identify Key Stakeholders and Challenges**
   - Identify all the stakeholders involved in the system.
   - Explain the roles and responsibilities of each stakeholder.
   - Discuss the major challenges and issues faced by the stakeholders.
3. **Analyze the Current Approach**
   - Describe the existing system or current process.
   - Identify its strengths and limitations.
   - Analyze the problems or gaps that need to be addressed.
4. **Propose a Suitable Solution Using Software Engineering Principles**
   - Design a software-based solution for the given problem.
   - Describe the major functional modules of the proposed system.
   - Identify the functional and non-functional requirements.
   - Explain how software engineering principles such as modularity, scalability, maintainability, reliability, usability, and security are incorporated into your solution.
5. **Justify the Chosen Methodology/Framework**

- Select an appropriate Software Development Life Cycle (SDLC) model or software development framework (e.g., Agile, Scrum, Waterfall, Spiral, DevOps).
- Justify why the selected methodology/framework is suitable for the proposed system.
- Explain how it supports successful project development and maintenance.

6. **Discuss Expected Outcomes and Limitations**
   - Describe the expected benefits of the proposed solution.
   - Explain how the solution addresses the identified challenges.
   - Discuss potential risks, implementation challenges, and limitations.
   - Suggest possible future enhancements.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Understanding (30%)	Clearly explains the problem, objectives, and scope with complete understanding.	Explains the problem with minor omissions.	Basic understanding with limited explanation.	Poor understanding; objectives and scope are unclear or missing.
Stakeholder & Challenge Analysis (25%)	Identifies all stakeholders, their roles, and challenges with clear analysis.	Identifies most stakeholders and challenges with adequate analysis.	Identifies only a few stakeholders or challenges.	Stakeholders and system analysis are incomplete or inaccurate.
Current System Analysis (20%)	Thoroughly analyzes the existing system, strengths, weaknesses, and identifies all gaps.	Good analysis with most strengths and weaknesses identified.	Limited analysis with partial identification of issues.	Proposed solution is unclear or inappropriate; requirements are largely missing.
Proposed Software Solution & Software Engineering Principles (15%)	Proposes an innovative, feasible solution applying appropriate software engineering principles (modularity, scalability, maintainability, security, etc.).	Suitable solution with good application of software engineering principles.	Basic solution with limited application of software engineering principles.	Methodology is inappropriate, poorly justified, or not discussed.
Methodology/ Framework Justification	Selects the most suitable SDLC/model/fram	Suitable methodology selected with	Methodology selected but justification is	Outcomes, limitations, or future enhancements

(10%)	ework and provides strong technical justification.	adequate justification.	weak.	are missing; report lacks organization and clarity.

Criteria	**Mark**
Problem Understanding (30%)	/5
Stakeholder & Challenge Analysis (25%)	/5
Current System Analysis (20%)	/5
Proposed Software Solution & Software Engineering Principles (15%)	/5
Methodology/ Framework Justification (10%)	/5
TOTAL	/25

Annexure A – Case Study Analysis

Problem Statement 7: Smart School Bus Tracking and Student Safety System

Student: *M. Rudresh* ***Reg. No.:*** *192372339* ***Date:*** *10-10-2026*

1. Understand the Problem Statement

1.1 The problem in my own words

Schools carry students on dedicated buses every day, yet most of them still run this service almost blind. Parents do not know where a bus is or when it will arrive, the school does not know instantly whether a child has boarded or got off at the right stop, and a detour, breakdown or long delay is usually discovered only when a worried parent phones. The task is to build a **Smart School Bus Tracking and Student Safety System** that follows each bus in real time, manages routes, records every boarding and drop-off event, and shows authorised parents and school staff accurate transport information. It must also raise automatic alerts for route deviations, unexpected delays and other important events, and keep a central record of everything that happens.

1.2 Objectives of the proposed system

- Maintain student transportation profiles (route, stops, guardians).
- Manage bus and driver information (registration, capacity, licences, document expiry).
- Define transportation routes with ordered stops and schedules.
- Track buses in real time using GPS.
- Record student boarding and drop-off events.
- Notify parents about bus status (departure, nearing stop, boarding, drop-off, delay).
- Detect route deviations automatically.
- Record and track transportation incidents to closure.
- Generate transportation reports for administrators.

1.3 Scope of the problem

In scope: daily home-to-school and school-to-home transport on school buses; live tracking; student attendance on the bus; route and stop management; parent, driver and administrator interfaces; alerts; incident log; reports. **Out of scope (for this release):** fee collection, payroll, vehicle maintenance scheduling, and field trips outside the defined routes. Hardware (GPS trackers, RFID or QR readers) is treated as an input device whose data the software consumes.

2. Identify Key Stakeholders and Challenges

Stakeholder	Roles and responsibilities	Major challenges
Parents / guardians	Check bus location and ETA, receive alerts, update contact details, report concerns.	No live visibility; uncertainty at the stop; worry if a child is late; late or no information on incidents.
Students	Board and leave the bus at the assigned stops; carry the ID or RFID card.	Risk of being missed, left behind or dropped at the wrong stop; long waits.
School administration	Own the transport policy; monitor the fleet; review reports; approve routes and buses.	No single view of all buses; slow response to incidents; liability and parent complaints; scattered records.
Transport coordinator	Plan routes and stops, assign students, buses and drivers, handle deviations and delays.	Manual route changes; hard to find which bus is where; many phone calls during disruptions.
Drivers	Follow the route and schedule, start and end trips, report incidents.	Traffic and breakdowns; pressure of schedule; extra reporting work; limited tech skills.
Bus attendants	Mark student boarding and drop-off, supervise behaviour on board.	Paper registers; crowded stops; scanning delays; errors in manual marking.

Stakeholder	Roles and responsibilities	Major challenges
IT / system administrator	Run and secure the platform, manage user accounts, back up data.	Data privacy, uptime, device integration, scaling to more buses.
Regulators and emergency services	Set safety rules; respond to accidents (RTO, police, hospitals).	Need quick, reliable location and incident information for compliance and rescue.

Overall challenges: real-time data must be accurate and fast on mobile networks; child data is sensitive and needs strong privacy; the interfaces must suit users with very different skills; and the system must cope with GPS noise, poor connectivity and daily route changes.

3. Analyze the Current Approach

3.1 Existing system or process

In most schools today buses run on a fixed timetable. Students are assigned to routes in spreadsheets or registers. Drivers and attendants mark attendance on paper (or not at all), parents call the driver or school office to ask where the bus is, and problems are reported by phone or messaging groups. Some schools fit basic GPS trackers, but these often show only a dot on a map to staff, without student-level data or parent alerts.

3.2 Strengths and limitations

Strengths of the current process	Limitations of the current process
Simple and cheap; no technology needed. Familiar to drivers, attendants and parents. Personal contact with the driver in an emergency. Flexible for small changes on the day.	No real-time location or ETA for parents. Boarding and drop-off not verified or time-stamped. Deviations and delays noticed late and by chance. Records are on paper, lost or inconsistent. No audit trail or reports; hard to prove what happened. Depends on individual people remembering to call.

3.3 Gaps that need to be addressed

Gap	Effect on safety or service
No live tracking and ETA	Parents wait uncertainly; the school cannot see where its buses are.
No verified boarding and drop-off record	A child can be missed or dropped wrongly without anyone knowing quickly.
No automatic deviation, delay or stop alerts	Slow response to detours, breakdowns and emergencies.
Data scattered across paper, sheets and chats	No single source of truth; duplicated and conflicting data.
No structured incident management	Weak follow-up, no accountability, no trend analysis.
No reporting or analytics	Routes and drivers cannot be improved using evidence.

4. Propose a Suitable Solution Using Software Engineering Principles

4.1 Software-based solution

The proposed solution is a web and mobile platform with a central server. A GPS tracker on each bus sends its position every 5 to 10 seconds. Attendants scan student RFID or QR cards (or tap in the attendant app) at each stop. The server stores this data, calculates ETAs, compares the live position with the planned route, and sends notifications. Parents use a mobile app; administrators and coordinators use a web dashboard; drivers and attendants use a simple trip app.

Figure 1. Layered architecture of the proposed system

Client layer	Parent mobile app	Admin / coordinator web dashboard	Driver and attendant app
Service layer (REST API with login and roles)	Tracking and ETA service	Route and profile service	Attendance service
	Deviation and alert engine	Notification service (push, SMS, e-mail)	Incident and reporting service
Data layer	PostgreSQL + PostGIS database	Cache and message queue	Backups and audit log
Field devices	GPS tracker on each bus	RFID / QR card reader	Driver or attendant phone

4.2 Major functional modules

Module	Function
Student profile	Student, guardian, route and stop assignment, ID card mapping.
Bus and driver management	Bus, driver and attendant records; assignment; licence and insurance expiry reminders.
Route management	Create routes with ordered stops, timings and a planned path.
Live tracking	Receive GPS points, show buses on a map, calculate ETA.
Boarding and drop-off	Record each scan with time, stop and bus; show the on-board list.
Parent notification	Send alerts only to the guardians of affected students.
Deviation detection	Raise alerts for route deviation, long stops, delay, overspeed and signal loss.
Incident management	Log accidents, breakdowns and complaints with severity and status history.
Reporting	Attendance, punctuality, deviation, incident and fleet reports with PDF and Excel export.

Deviation logic: each route is stored as a polyline. For every GPS point the system finds the shortest distance to that line; if the bus stays more than about 150 m away for 3 consecutive readings, a deviation alert is raised and closed automatically when the bus returns. ETA = current time + remaining distance ÷ recent average speed + expected stop time.

4.3 Functional and non-functional requirements

Functional requirements	Non-functional requirements
FR1 Manage students, buses, drivers and routes. FR2 Assign each student to a route and stops. FR3 Show live bus location every 5 to 10 s. FR4 Record boarding and drop-off with time. FR5 Calculate and show ETA at each stop. FR6 Detect route deviation, delay, long stop. FR7 Send notifications to authorised parents. FR8 Log and track incidents. FR9 Generate and export reports. FR10 Role-based login and access control.	**Performance:** position visible within 10 s. **Availability:** at least 99.5% in transport hours. **Scalability:** 200 buses and 10,000 students. **Security:** HTTPS, encryption, audit log. **Usability:** simple screens, low-end phones. **Reliability:** GPS device buffers data offline. **Maintainability:** modular, documented, tested code.

4.4 How software engineering principles are applied

Principle	How it is incorporated
Modularity	Nine loosely coupled modules with clear interfaces; each can be built, tested and replaced independently.
Scalability	Stateless API behind a load balancer, message queue for GPS data, and database indexing on trip and time columns, so more buses can be added without redesign.

Principle	How it is incorporated
Maintainability	Layered architecture, coding standards, version control, automated tests and documentation make changes and fixes easy.
Reliability	Offline buffering in the GPS device, retry of failed notifications, SMS fallback, database backups and a health-monitoring alert.
Usability	Role-specific, minimal screens; one-tap actions for attendants; a map and ETA for parents in local language options.
Security	Role-based access, hashed passwords, token sessions, HTTPS, encrypted personal data, location shown only during active trips, audit logs.

5. Justify the Chosen Methodology / Framework

5.1 Selected model: Agile (Scrum)

I select **Agile with the Scrum framework**, delivered in 2-week sprints, supported by DevOps practices (version control, continuous integration and automated deployment).

Model	Fit for this project
Waterfall	Needs all requirements fixed upfront; users (parents, drivers) only see the product at the end; changes are costly. Poor fit.
Spiral	Good for risk control but heavy on documentation and cost for a school-sized project. Partial fit.
DevOps alone	Strong for deployment and operations but is not a requirements and planning method. Used as a supporting practice.
Agile (Scrum)	**Short cycles, early working software, frequent feedback, easy to change priorities. Best fit.**

5.2 Why it is suitable

- **Requirements will evolve:** parents, drivers and administrators will ask for changes after seeing the first version (for example alert timings or screens).
- **Early value and early risk reduction:** live tracking, the riskiest technical part, is built and field-tested in the first sprints.
- **Incremental delivery:** each sprint produces a usable increment that can be piloted on one real bus route.
- **Continuous feedback:** sprint reviews with the transport coordinator and a parent representative keep the product aligned with real needs.
- **Quality and safety focus:** automated tests and a definition of done in every sprint reduce defects in a safety-critical product.

5.3 Sprint plan

Sprint	Goal (backlog items)
0	Requirements, product backlog, architecture, database design, tool setup, CI pipeline.
1	Login and roles, student, bus, driver and route management.
2	GPS ingestion, live map and ETA (pilot on one bus).
3	Boarding and drop-off recording, on-board list.
4	Parent app, notifications, route deviation and delay alerts.
5	Incident management, reports, security hardening and performance testing.
6	User acceptance testing, pilot on several routes, training and go-live.

5.4 Support for development and maintenance

A prioritised product backlog lets the team always work on the most valuable items. Daily stand-ups, sprint reviews and retrospectives expose problems early and improve the process. After go-live, the same backlog and sprint rhythm handle bug fixes, new bus routes and new features, while CI/CD pipelines and automated regression tests make each release small, safe and reversible.

6. Discuss Expected Outcomes and Limitations

6.1 Expected benefits and how challenges are addressed

Challenge	How the solution addresses it	Expected benefit
No visibility for parents	Live map, ETA and event alerts in the parent app	Better parental visibility; less uncertainty
Child missed or dropped wrongly	Scan-based boarding and drop-off with missed drop-off alert	Improved student safety
Late discovery of deviations or delays	Automatic deviation, delay, long-stop and signal-lost alerts	Faster response to incidents
Scattered paper records	One central database with audit log	Centralized transportation records
Weak incident follow-up	Incident log with status and time stamps	Accountability and measurable response time
Poor route planning data	Punctuality, deviation and utilisation reports	Improved route management

6.2 Risks, implementation challenges and limitations

Risk / limitation	Mitigation
GPS inaccuracy in tunnels or dense areas; network gaps delay updates	Offline buffering, smoothing of readings, 3-reading rule before alerts.
Staff forgetting to scan students	Simple one-tap or card scan, attendant training, report on missed scans.
Privacy and misuse of child location data	Role-based access, encryption, location visible only during active trips, retention policy.
Hardware cost and device failure	Phased roll-out, low-cost trackers, spare units, health monitoring.
Resistance to change by drivers and parents	Training, pilot routes, feedback in every sprint.
SMS or push delivery failures	Retry logic, channel fallback, delivery status tracking.

6.3 Possible future enhancements

- AI-based ETA and traffic-aware route optimisation.
- Face recognition or biometric boarding verification.
- Panic button and in-bus camera integration with live alerts.
- Vehicle maintenance, fuel and driver-behaviour analytics.
- Integration with the school's attendance and ERP systems.
- Multi-language support and a parent-facing web portal.

Conclusion: The Smart School Bus Tracking and Student Safety System replaces guesswork with live, verified data. Built using modular design and an Agile (Scrum) approach, it improves safety, gives parents confidence, gives the school control and gives everyone a single trustworthy record of every journey.